Software Engineering Lab

Lab 35

WebSocket vs Server-Sent Events

Memilih Protokol Real-Time Berdasarkan Directionality dan Kompleksitas



Lukman / lukman-ss

Dilema Protokol Real-Time Modern

Kesalahan pemilihan protokol memicu beban operasional dan kompleksitas stateful

GEJALA UMUM

Fitur notifikasi dan live dashboard sering otomatis dipasangi WebSocket
WebSocket memaksakan upgrade TCP, heartbeat kustom, dan reconnection manual
Proxy reverse dan load balancer memerlukan sticky routing yang rumit

BIAYA OPERASIONAL

Stateful connection membebani memori saat client hanya membaca data pasif
Kegagalan reconnection storm melumpuhkan gateway saat restart
Debugging framing biner lebih sulit dibanding stream teks standar

PERTANYAAN UTAMA

Kapan arsitektur benar-benar memerlukan pipa dua arah (bidirectional)?
Bagaimana memanfaatkan streaming HTTP standar tanpa overhead WebSocket?

Banyak tim rekayasa perangkat lunak menganggap WebSocket sebagai standar de facto untuk semua kebutuhan real-time. Padahal, memasang WebSocket untuk use case satu arah seperti notifikasi atau live dashboard memperkenalkan overhead operasional yang tidak perlu. SSE menyediakan mekanisme server push berbasis HTTP murni yang jauh lebih ramah proxy dan hemat resource.

Do not pay bidirectional connection tax for unidirectional server push.

Mental Model: Directionality Rule

Arah aliran pesan menentukan protokol, bukan sekadar latensi atau tren

SERVER-PUSH ONLY

Arah komunikasi: Server -> Client secara sepihak
Contoh: Notifikasi sistem, live stock ticker, streaming token LLM
Solusi tepat: Server-Sent Events (SSE) berbasis HTTP standar

FULL-DUPLEX BIDIRECTIONAL

Arah komunikasi: Server <-> Client secara independen dan simultan
Contoh: Chat interaktif, collaborative canvas, multiplayer game
Solusi tepat: WebSocket (RFC 6455) atas koneksi raw TCP

PEDOMAN ARSITEKTUR

Jika client tidak pernah mengirim pesan di channel yang sama: pilih SSE
Jika client harus push data dengan latensi sub-milidetik: pilih WebSocket

Keputusan arsitektural dapat disederhanakan melalui satu pertanyaan fundamental: apakah client perlu mengirim pesan ke server secara independen pada saluran yang sama? Jika komunikasi hanya satu arah dari server ke client, SSE memberikan solusi paling bersih. WebSocket hanya diwajibkan ketika komunikasi dua arah berlatensi rendah mutlak diperlukan.

Directionality dictates the protocol: server-push belongs to SSE, bidirectional belongs to WebSocket.

Topologi Arsitektur Lab 35

Unified HTTP Router di Go menghosting /sse, /ws, dan /publish secara murni

ENDPOINT DISPATCH

GET /sse -> SSE Hub (Handler streaming text/event-stream)
GET /ws -> WebSocket Upgrader (RFC 6455 Hijack + loop)
POST /publish -> Trigger broadcast event ke seluruh client SSE aktif

ZERO DEPENDENCIES

Implementasi 100% Go Standard Library (net/http, sync, crypto)
Tanpa gorilla/websocket atau pustaka eksternal pihak ketiga
Membuktikan mekanisme low-level framing dan HTTP connection hijacking

ISOLASI KOMPONEN

internal/sse: Hub broadcast dengan buffer history dan Last-Event-ID
internal/ws: RFC 6455 parser, unmasking XOR, frame text & binary

Lab 35 mendemonstrasikan kedua protokol di bawah satu HTTP ServeMux di internal/server/server.go. Pendekatan zero external dependencies membongkar bagaimana Go standard library mampu menangani streaming HTTP (SSE) melalui http.Flusher sekaligus protokol non-HTTP (WebSocket) melalui interface http.Hijacker.

Unified standard library architecture demonstrates both protocols side by side without third-party bias.

Anatomi Server-Sent Events (SSE)

Streaming teks persisten berbasis HTTP dengan MIME type text/event-stream

HEADER STANDAR HTTP

Content-Type: text/event-stream (wajib untuk browser EventSource)
Cache-Control: no-cache (mencegah caching di perantara)
Connection: keep-alive (menjaga socket TCP tetap terbuka)
X-Accel-Buffering: no (menonaktifkan buffering di Nginx proxy)

STRUKTUR FRAME EVENT

id: -> Identifier unik untuk resumption
event: -> Tipe custom event untuk event listener
retry: -> Durasi tunggu reconnect client browser
data: -> Payload teks (dapat multi-line dengan data: prefix)

DELIMITER BLOK

Setiap baris diakhiri \n tunggal
Akhir dari satu blok event ditandai dengan \n\n ganda

Spesifikasi WHATWG mendefinisikan SSE sebagai respons HTTP berdurasi panjang dengan format text/event-stream. Setiap event terdiri dari field id, event, retry, dan data yang diakhiri oleh double newline. Karena berbasis HTTP murni, SSE langsung kompatibel dengan infrastruktur web yang ada seperti TLS termination dan monitoring.

SSE is plain HTTP with structured text frames delimited by double newlines.

Format Event SSE: Multi-Line Framing

Implementasi method Event.Format() memastikan kepatuhan spesifikasi WHATWG

```
func (e Event) Format() []byte {
    var sb strings.Builder
    if e.ID > 0 { fmt.Fprintf(&sb, "id: %d\n", e.ID) }
    if e.Event != "" { fmt.Fprintf(&sb, "event: %s\n", e.Event) }
    if e.Retry > 0 { fmt.Fprintf(&sb, "retry: %d\n", e.Retry) }
    lines := strings.Split(e.Data, "\n")
    for _, l := range lines {
        fmt.Fprintf(&sb, "data: %s\n", l)
    }
    sb.WriteString("\n")
    return []byte(sb.String())
}
```

Method Format() pada struct Event mematuhi aturan WHATWG secara ketat. Jika payload mengandung baris baru, setiap baris harus diawali prefiks 'data: '. Kegagalan memecah baris baru akan merusak parsing client EventSource di browser. Karakter double newline di akhir menjadi sinyal bahwa frame event telah lengkap.

ATURAN MULTI-LINE

Payload data yang memiliki newline harus dipecah menjadi beberapa baris data:
Browser EventSource akan menggabungkan kembali baris-baris tersebut dengan newline
Trailing newline kedua menutup frame dan memicu event dispatch di client

Split multi-line data payloads across multiple data prefixes before terminating with double newline.

SSE Hub: Resumption via Last-Event-ID

State management memutar ulang event tertinggal saat client melakukan reconnect

MEKANISME RESUMPTION

Client terputus -> browser otomatis reconnect mengirim header Last-Event-ID
Hub membaca Last-Event-ID dan memfilter history: evt.ID > lastID
Event yang terlewat dikirimkan seketika sebelum mendengarkan stream live

```
func (h *Hub) Subscribe(lastID int) (chan Event, []Event, func()) {
    h.mu.Lock(); defer h.mu.Unlock()
    var replay []Event
    for _, evt := range h.history {
        if evt.ID > lastID { replay = append(replay, evt) }
    }
    ch := make(chan Event, 64)
    h.clients[ch] = struct{}{}
    return ch, replay, func() { ... }
}
```

JAMINAN KONSISTENSI

Client tidak kehilangan update penting akibat network flap sementara
Proses terjadi transparan tanpa perlu logika retry kustom di aplikasi

Fitur paling unggul dari SSE adalah kemampuan auto-reconnect bawaan yang dipadukan dengan header Last-Event-ID. Ketika koneksi putus, browser secara otomatis mengirimkan ID terakhir yang diterimanya. Hub di backend menyaring riwayat in-memory dan langsung memutar ulang event yang terlewat sebelum menyambungkan client ke antrean live.

Last-Event-ID is a built-in recovery offset: replay what was missed, then stream live.

Loop Streaming HTTP & Heartbeat

Flush berkala dan komentar keep-alive mencegah timeout reverse proxy

```
flusher, ok := w.(http.Flusher)
for {
    select {
    case <-ctx.Done(): return
    case evt, ok := <-ch:
        if !ok { return }
        w.Write(evt.Format())
        flusher.Flush()
    case <-ticker.C:
        w.Write([]byte(": keep-alive\n\n"))
        flusher.Flush()
    }
}
```

Koneksi SSE memerlukan http.Flusher agar frame teks terkirim ke client seketika tanpa tertahan di buffer Go runtime. Loop select memantau tiga hal: sinyal pemutusan dari context request, event baru dari broadcast channel, dan ticker 15 detik yang mengirim komentar ': keep-alive' untuk mencegah proxy memutus koneksi akibat idle timeout.

PERAN FLUSHER

http.Flusher mendorong buffer HTTP kernel seketika tanpa menunggu penuh
Mencegah penumpukan paket di TCP socket buffer

KOMENTAR KEEP-ALIVE

Prefix titik dua (':') diabaikan oleh parser EventSource browser
Mengirim data setiap 15 detik untuk menjaga state NAT/Firewall proxy tetap

Flushing pushes frames instantly; comment frames defeat proxy idle timeouts.

Anatomi Protokol WebSocket (RFC 6455)

Protokol non-HTTP full-duplex melalui upgrade TCP connection tunggal

HANDSHAKE HTTP/1.1

Request: GET /ws dengan Upgrade: websocket & Connection: Upgrade
Sec-WebSocket-Key: Nilai random 16-byte yang di-encode base64
Respons: HTTP/1.1 101 Switching Protocols
Sec-WebSocket-Accept: base64(SHA-1(Key + WS_GUID))

KONEKSI NON-HTTP

Setelah status 101, socket dilepas dari HTTP engine (Hijacked)
Komunikasi beralih ke framing biner raw TCP berarah ganda
Server dan client dapat mengirim frame kapan saja secara simultan

TIPE PAYLOAD

Mendukung teks UTF-8 murni (Opcode 0×1)
Mendukung raw binary byte array (Opcode 0×2)
Control frames: Close (0×8), Ping (0×9), Pong (0×A)

WebSocket dirancang oleh IETF RFC 6455 untuk menghadirkan kanal dua arah berlatensi rendah. Koneksi diawali dengan request HTTP biasa yang meminta Upgrade. Server merespons dengan 101 Switching Protocols dan menghitung Sec-WebSocket-Accept menggunakan GUID standar. Setelah itu, socket sepenuhnya beralih ke parsing frame biner.

WebSocket begins as HTTP handshake then metamorphoses into raw TCP binary framing.

Handshake Upgrade & Connection Hijacking

Mengambil alih raw socket TCP dari HTTP Server menggunakan http.Hijacker

```
h := sha1.New()
h.Write([]byte(key + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11"))
accept := base64.StdEncoding.EncodeToString(h.Sum(nil))

hj, ok := w.(http.Hijacker)
netConn, bufrw, err := hj.Hijack()
resp := "HTTP/1.1 101 Switching Protocols\r\n" +
    "Upgrade: websocket\r\nConnection: Upgrade\r\n" +
    "Sec-WebSocket-Accept: " + accept + "\r\n\r\n"
bufrw.WriteString(resp); bufrw.Flush()
```

Fungsi Upgrade() mendemonstrasikan kalkulasi kriptografis Sec-WebSocket-Accept sesuai RFC 6455. Dengan memanggil w.(http.Hijacker).Hijack(), Go melepaskan kendali socket TCP dari net/http runtime. Server menulis respons 101 Switching Protocols secara langsung ke buffer TCP dan membungkus socket ke dalam struct Conn kustom.

IMPLIKASI HIJACK

net/http tidak lagi mengelola timeout atau penutupan socket
Aplikasi bertanggung jawab penuh atas buffer I/O dan life cycle koneksi

Hijacking frees the raw TCP socket from net/http lifecycle constraints.

Struktur Frame Biner RFC 6455

Encoding header hemat byte dengan mask flag, opcode, dan multi-tier length

LAYOUT FRAME RFC 6455

Byte 0: FIN (1 bit) + RSV (3 bits) + Opcode (4 bits)
Byte 1: MASK (1 bit) + Payload Length (7 bits)
Extended Length (opsional): 2 bytes (jika len=126) atau 8 bytes (jika len=127)
Masking Key: 4 bytes (wajib ada untuk frame dari client ke server)
Payload Data: N bytes (dimasking dengan XOR jika MASK=1)

ENCODING PANJANG PAYLOAD

<= 125 bytes -> Cukup 7 bit pada Byte 1
126 - 65,535 -> 7 bit bernilai 126 + 2 byte uint16 BigEndian
> 65,535 bytes -> 7 bit bernilai 127 + 8 byte uint64 BigEndian

ATURAN MASKING

Client -> Server: Wajib dimasking (mencegah cache poisoning perantara)
Server -> Client: Dilarang dimasking (RFC 6455 Section 5.1)

Struktur frame RFC 6455 dirancang sangat efisien. Header terkecil hanya membutuhkan 2 byte. Panjang payload dikodekan secara adaptif: 7 bit untuk pesan kecil, 16 bit untuk ukuran menengah, dan 64 bit untuk data raksasa. Mekanisme masking wajib diterapkan oleh client untuk mencegah eksploitasi cache poisoning pada proxy perantara.

Bit-level efficiency: 2-byte minimum header accommodates small frames to gigabytes.

Parsing Frame & Operasi XOR Masking

Membaca byte header dan memulihkan payload asli client secara bitwise

```
var head [2]byte
io.ReadFull(c.rw, head[:])
opcode = head[0] & 0x0F
masked := (head[1] & 0x80) != 0
length := uint64(head[1] & 0x7F)
// ... handle extended length ...
if masked {
    io.ReadFull(c.rw, mask[:])
    io.ReadFull(c.rw, data)
    for i := range data {
        data[i] ^= mask[i%4] // Bitwise XOR unmasking
    }
}
```

Method ReadFrame() mengekstrak header biner dari stream TCP. Jika bit MASK aktif, server membaca 4 byte masking key. Setiap byte payload kemudian di-XOR dengan mask[i % 4] untuk membuka enkripsi transport. Sifat simetris XOR memungkinkan decoding langsung di memori secara cepat tanpa alokasi buffer sekunder.

SIFAT SIMETRIS XOR

Operasi (A ^ B) ^ B = A
Proses unmasking di server identik dengan proses masking di client
Biaya CPU rendah namun membutuhkan iterasi byte-by-byte

XOR unmasking restores wire bytes in-place: data[i] ^= mask[i % 4].

labs/35-websocket-and-sse/internal/ws/ws.go:84-127

WriteFrame: Konstruksi Frame Outbound

Serialisasi data text dan biner dengan pengaturan bit FIN dan masking

```
func (c *Conn) WriteFrame(opcode byte, payload []byte, mask bool) error {
    c.mu.Lock(); defer c.mu.Unlock()
    var buf []byte
    b0 := byte(0x80) | (opcode & 0x0F) // FIN=1
    buf = append(buf, b0)
    // ... encode length (7, 16, or 64 bit) ...
    if mask {
        // append maskKey and maskedPayload
    } else {
        buf = append(buf, payload...)
    }
    _, err := c.rw.Write(buf)
    return err
}
```

Method WriteFrame() menyusun paket biner untuk dikirim ke socket. Mutex c.mu menjamin bahwa penulisan frame bersifat atomic sehingga dua goroutine tidak mencampur aduk byte frame di TCP stream. Bit FIN diset ke 1 untuk menandakan pesan mandiri tanpa fragmentasi lanjutan.

THREAD-SAFETY SOCKET

Mutex memastikan frame tidak terfragmentasi saat ditulis bersamaan
Bit FIN=1 menandai frame lengkap (tidak terpecah dalam continuation)

Mutex synchronization guarantees atomic frame delivery over raw TCP sockets.

Server Loop: Echo Teks & Reversal Biner

Menangani OpClose, OpPing, OpText, dan OpBinary pada endpoint /ws

```
switch op {
case ws.OpClose:
    conn.WriteFrame(ws.OpClose, nil, false); return
case ws.OpPing:
    conn.WriteFrame(ws.OpPong, data, false)
case ws.OpText:
    conn.WriteFrame(ws.OpText, append([]byte("echo: "), data...), false)
case ws.OpBinary:
    rev := make([]byte, len(data))
    for i := range data { rev[i] = data[len(data)-1-i] }
    conn.WriteFrame(ws.OpBinary, rev, false)
}
```

Loop pemrosesan WebSocket di server menangani berbagai tipe frame kontrol dan data. Untuk teks, server menambahkan prefiks 'echo: '. Untuk biner, payload dibalik urutan byte-nya sebelum dikirim kembali. Reversal ini memberikan bukti deterministik bahwa protokol mampu mentransfer biner murni tanpa kompromi.

PEMBUKTIAN BINER

Client mengirim 0×DEADBEEF -> Server membalikkan menjadi 0×EFBEADDE
Membuktikan integritas byte murni tanpa distorsi encoding karakter

Byte reversal conclusively verifies true binary transport fidelity.

Eksekusi Demo: Bukti Terminal Langsung

Output deterministik dari cmd/demo memvalidasi resumption dan duplexing

```
> 1. SSE: Unidirectional Server Push with Last-Event-ID
Client connected with Last-Event-ID: 1. Received resumed events:
  [SSE Stream] id: 2
  [SSE Stream] event: news
  [SSE Stream] retry: 2000
  [SSE Stream] data: Update: Faster routing in net/http
```

```
> 2. WebSocket: Full-Duplex Text and Binary Protocol
Client sending WS text frame: "Hello RFC 6455"
Server responded with Opcode 0x1 (Text): "echo: Hello RFC 6455"
Client sending WS binary frame: DEADBEEF
Server responded with Opcode 0x2 (Binary): EFBEADDE
Binary frame payload verified successfully.
```

Eksekusi cmd/demo membuktikan klaim teknis secara nyata. Pada bagian SSE, pengiriman Last-Event-ID: 1 berhasil mengabaikan event 1 dan memutar ulang event 2. Pada bagian WebSocket, frame teks di-echo dengan benar dan frame biner DEADBEEF berhasil dibalik menjadi EFBEADDE tanpa kecacatan data.

Deterministic verification: SSE resumes exactly from offset; WebSocket mirrors bytes exactly.

Hasil Pengujian Test Suite & Concurrency

4 test case diverifikasi lolos dengan Race Detector aktif (Go 1.22)

```
=== RUN   TestSSE_Formatting            -> PASS (0.00s)
=== RUN   TestSSE_LastEventID_Resumption -> PASS (0.00s)
=== RUN   TestWebSocket_TextAndBinary    -> PASS (0.00s)
=== RUN   TestSSE_Concurrency           -> PASS (0.05s)
PASS: go test -race ./... (1.559s - Race Clean)
```

Seluruh rangkaian pengujian di tests/protocol_test.go lulus dengan Go Race Detector aktif. Test concurrency membuktikan bahwa non-blocking broadcast channel pada Hub mampu melayani 10 client paralel saat menerima 5 event acak tanpa terjadi data race atau goroutine starvation.

SKENARIO CONCURRENCY

10 client subscriber terhubung secara paralel ke SSE Hub
5 event broadcast dipicu secara simultan di goroutine terpisah
Mutex dan unbuffered select mencegah deadlock maupun blocking antar client

DETERMINISTIK FORMATTING

Test memvalidasi ordering field id, event, retry, data, dan delimiter ganda

Race-clean test pass proves thread-safe synchronization across concurrent streams.

Matriks Evaluasi: WebSocket vs SSE

Perbandingan mendalam 8 dimensi arsitektur dan operasional

Dimensi	WebSocket (RFC 6455)	Server-Sent Events (SSE)
Arah Komunikasi	Full-Duplex (2 Arah)	Unidirectional (Server->Client)
Transport / Pipa	Raw TCP (setelah Upgrade)	Standard HTTP Streaming
Tipe Payload	UTF-8 Teks & Raw Biner	UTF-8 Teks Murni Saja
Auto-Reconnect	Manual / Custom Client	Bawaan Browser EventSource
Resumption State	Tidak ada di spec level	Bawaan via Last-Event-ID
Proxy / LB Compat	Butuh Sticky / WS Bypass	Standar HTTP (No buffering)
HTTP/2 Multiplex	Butuh RFC 8441 CONNECT	Native HTTP/2 Multiplexing

Matriks perbandingan memperjelas trade-off arsitektural. WebSocket menawarkan kemampuan full-duplex dan dukungan biner dengan kompensasi hilangnya auto-reconnect bawaan dan meningkatnya kerumitan proxy. SSE memberikan ketahanan jaringan otomatis dan dukungan native HTTP/2 stream multiplexing.

RINGKASAN TRADE-OFF

WebSocket unggul dalam fleksibilitas payload dan interaktivitas upstream

WebSocket optimizes for protocol capability; SSE optimizes for operational simplicity.

Tantangan Proxy, Firewall & Gateway

Bagaimana middleware jaringan memperlakukan koneksi persisten

ISU PROXY PADA WEBSOCKET

Load balancer modern (seperti AWS ALB) butuh sticky sessions jika node memegang state in-memory
Beberapa firewall perusahaan memblokir request non-HTTP setelah Upgrade
Idle timeout TCP memutus socket jika tidak ada traffic Ping/Pong aktif

ISU PROXY PADA SSE

Buffering di perantara: Nginx menahan chunk response jika proxy_buffering aktif
Solusi: Tambahkan header X-Accel-Buffering: no pada respons server
Batas koneksi HTTP/1.1 browser (maks 6 koneksi per domain) sebelum HTTP/2

REKOMENDASI PRODUKSI

Deploy SSE di atas HTTP/2 untuk menghilangkan batas koneksi domain
Terapkan distributed message broker (Redis Pub/Sub) untuk scaling multi-node

Koneksi jangka panjang rentan terhadap timeout dan pemotongan oleh proxy perantara. WebSocket kerap terhambat oleh firewall korporat yang mencurigai protokol non-HTTP. SSE menghadapi risiko proxy buffering di Nginx, yang dapat diselesaikan dengan menyetel X-Accel-Buffering: no atau menonaktifkan proxy_buffering.

Reverse proxies buffer HTTP unless instructed otherwise; firewalls distrust protocol upgrades.

Dampak HTTP/2 & Multiplexing Aliran

Mengapa evolusi HTTP memperkuat posisi Server-Sent Events

SSE NATIVE DI HTTP/2

SSE berjalan di atas stream HTTP/2 standar tanpa ekstensi
Multiplexing memungkinkan puluhan stream SSE berbagi 1 koneksi TCP fisik
Mengeliminasi batas browser 6 koneksi per origin pada HTTP/1.1

WEBSOCKET DI HTTP/2 (RFC 8441)

HTTP/2 melarang header Connection: Upgrade (RFC 9113)
WebSocket di HTTP/2 memerlukan Extended CONNECT method (:protocol=websocket)
Dukungan server dan proxy gateway masih belum universal di industri

KEUNGGULAN OPERASIONAL

SSE secara natural mendapatkan kompresi header HPACK dan flow control HTTP/2

Revolusi HTTP/2 secara dramatis meningkatkan daya tarik SSE. Melalui stream multiplexing, puluhan saluran SSE dapat berbagi satu koneksi TCP tanpa batasan 6 koneksi per domain. Sebaliknya, WebSocket memerlukan implementasi RFC 8441 yang rumit karena HTTP/2 melarang mekanisme connection upgrade tradisional.

HTTP/2 natively supercharges SSE while demanding complex extensions for WebSocket.

Studi Kasus Nyata: Notifikasi & AI Tokens

Membedah pilihan arsitektur OpenAI, Claude, dan Sistem Notifikasi Skala Besar

STREAMING TOKEN LLM (OPENAI / ANTHROPIC)

Karakteristik: Server menghasilkan token kata demi kata secara real-time
Arah: 100% Server -> Client setelah prompt awal dikirim
Pilihan arsitektur: Server-Sent Events (SSE) adalah standar industri universal

KOLABORASI DOKUMEN & CHAT (FIGMA / SLACK)

Karakteristik: Kursor user bergerak cepat, pesan teks dikirim dua arah
Arah: Bidirectional interaktif dengan latensi minimal
Pilihan arsitektur: WebSocket dengan enkripsi frame biner

PEMBELAJARAN ARSITEKTURAL

Penggunaan SSE pada API LLM membuktikan ketahanan dan kesederhanaan protokol

Tren modern membuktikan penerapan aturan directionality. Layanan AI generative seperti OpenAI ChatGPT memilih SSE untuk men-stream token kata demi kata ke browser karena request prompt sudah selesai di awal. Sebaliknya, aplikasi seperti Figma atau multiplayer gaming mengandalkan WebSocket karena user terus-menerus mengirimkan data interaktif.

Match real-world traffic patterns: LLM tokens stream via SSE; multi-user editing flows via WebSocket.

Batasan Model & Implementasi Lab 35

Penyederhanaan pedagogis untuk menonjolkan prinsip mekanika protokol

BATAS 1: WEBSOCKET FRAMING

Masking key client menggunakan nilai konstan (bukan crypto-random)
Belum mendukung fragmented continuation frames (OpContinuation 0×0)
Payload > 65,535 bytes memiliki parser BigEndian namun belum diuji ekstensif

BATAS 2: SSE HUB BUFFER

History event disimpan dalam slice in-memory tak terbatas (unbounded buffer)
Produksi memerlukan ring buffer dengan batas retensi atau Redis Streams

BATAS 3: SCALE & DISTRIBUTED STATE

Uji coba berjalan pada single node HTTP server tanpa distributed message bus

Lab ini dirancang sebagai sarana edukasi teknis murni. Masking key diset statis untuk memudahkan debugging inspeksi byte, dan history SSE disimpan dalam memori tanpa penggusuran. Dalam lingkungan produksi riil, implementasi memerlukan ring buffer berkapasitas tetap, masking acak kriptografis, dan distributed broker.

A lab isolates protocol mechanics; production demands ring buffers, random masks, and brokers.

Checklist Keputusan Arsitektur Real-Time

8 langkah verifikasi sebelum menentukan protokol transport aplikasi

EVALUASI KEBUTUHAN

[ ] Apakah client perlu mengirim pesan upstream di socket yang sama? (Ya->WS, Tidak->SSE)
[ ] Apakah data mengandung payload biner murni (audio/protobuf)? (Ya->WS)
[ ] Apakah sistem berjalan di balik corporate proxy / strict firewall? (Pilih SSE)
[ ] Apakah pemulihan event otomatis (reconnect) diperlukan secara gratis? (Pilih SSE)

VERIFIKASI PRODUKSI

[ ] Pastikan Nginx menonaktifkan proxy_buffering untuk SSE endpoint
[ ] Pasang heartbeat / ping-pong berkala untuk mencegah idle connection drops
[ ] Atur limit OS file descriptor (ulimit -n) untuk menangani ribuan koneksi
[ ] Siapkan Redis Pub/Sub untuk replikasi broadcast lintas cluster node

Gunakan checklist ini dalam tahap perancangan sistem real-time. Jika komunikasi upstream dapat dipenuhi melalui HTTP POST standar dan client hanya mendengarkan event, SSE adalah pilihan terbaik. Pastikan konfigurasi reverse proxy telah disesuaikan dan kapasitas file descriptor sistem operasi memadai.

Verify directionality, payload type, and proxy topology before committing to a protocol.

Tiga Prinsip Inti Komunikasi Real-Time

Ringkasan aksioma untuk membangun sistem streaming yang efisien

1. DIRECTIONALITY IS KING

Jangan bayar overhead koneksi dua arah jika trafik hanya mengalir satu arah
SSE adalah peluru perak untuk live notifications, feed, dan token streaming

2. LEVERAGE PLATFORM PRIMITIVES

Manfaatkan EventSource dan Last-Event-ID bawaan browser untuk auto-healing
Gunakan HTTP/2 multiplexing untuk menghemat handshake TCP koneksi

3. PROTOCOL RESPECTS BOUNDARIES

WebSocket adalah pilihan mutlak ketika interaktivitas biner dua arah dibutuhkan
Keduanya saling melengkapi, bukan saling meniadakan

Sistem real-time yang elegan memilih protokol berdasarkan karakteristik data transport. Hindari godaan over-engineering dengan memasang WebSocket di setiap use case. Dengan memilih SSE untuk pola satu arah dan WebSocket untuk pola interaktif dua arah, sistem tetap efisien, mudah di-scale, dan tahan banting.

Right protocol for the right direction: simplicity scales better than unnecessary duplexing.

Eksperimen kecil, eliminasi latensi tersembunyi.

Directionality dictates the protocol: server-push belongs to SSE, bidirectional belongs to WebSocket.

Software Engineering Lab 35 - WebSocket vs Server-Sent Events

internal/sse • internal/ws
internal/server • cmd/demo
tests/protocol_test.go • README.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab`

`labs/35-websocket-and-sse`

Lukman / lukman-ss

Sumber: commit b48fe6c3
Semua test PASS dengan Go 1.22 Race Detector aktif
Implementasi murni zero external dependencies